

**GHANA COMMUNICATION TECHNOLOGY
UNIVERSITY (GCTU)**



**FACULTY OF COMPUTING AND INFORMATION
SYSTEMS (FOCIS)
DEPARTMENT OF INFORMATION TECHNOLOGY**

**TITLE:
STUDENT ACADEMIC PROGRESS TRACKING SYSTEM**

A Project Work Submitted in Partial Fulfillment of the Requirements For
BSc. Information Technology

BY:

Richard Yawlui (4211231018)
Addo Foster Keteku (4211231161)

SUPERVISOR:

MR. FRANK BOATENG

FEBRUARY, 2026

DECLARATION

This project is presented as part of the requirements for BSc. Information Technology awarded by Ghana Communication Technology University. We hereby declare that this project is entirely the result of our own hard work, research, and enquiries. We are confident that this project work is not copied from any other person. All sources of information have however been acknowledged with due respect.

AUTHOR: Richard Yawlui

SIGNATURE

STUDENT ID: 4211231018

DATE:

AUTHOR: Addo Foster Keteku

SIGNATURE

STUDENT ID: 4211231161

DATE:

SUPERVISOR: MR. FRANK BOATENG

SIGNATURE

DATE:

HOD:

SIGNATURE

DATE:

ABSTRACT

University students monitor their academic progress through a scattered collection of notebooks, calculator applications and calendar reminders, none of which communicate with one another. The result is a fragmented record that makes it difficult to establish a current academic standing, to observe performance trends, or to relate study effort to examination outcomes. This project designed and developed a Student Academic Progress Tracking System: a web-based application that consolidates course management, assessment recording, weighted grade and grade point average computation, study session logging, revision through flashcards, document storage and goal tracking into a single interface. Requirements were established through a review of the literature on self-regulated learning, interviews with students and lecturers, and an examination of existing tracking tools; the system was then built using the Waterfall model, with HTML5, CSS3 and JavaScript on the client and PHP on the server, and is hosted locally on the Apache and MySQL services supplied with XAMPP. Each student signs in to an account created by an administrator; passwords are stored only as bcrypt hashes, every database query is filtered by the account held in the session, and no student can reach another student's records. Functional, usability, security and performance testing confirmed that grade calculations are accurate, that records persist across sessions and devices, that the access controls hold, and that the interface loads and responds well within the three-second target on both desktop and mobile displays. The system demonstrates that a purpose-built, zero-cost academic tracker, deployable by a department on a single machine, can give students the integrated overview that general-purpose productivity tools do not provide.

Keywords: academic progress tracking, self-regulated learning, grade point average, data visualisation, relational database, authentication, XAMPP.

TABLE OF CONTENTS

Declaration.....	ii
Abstract.....	iii
Table of Contents.....	iv
List of Tables.....	vi
List of Figures.....	vii
List of Listings.....	viii
List of Abbreviations.....	ix
Acknowledgement.....	x
CHAPTER ONE: INTRODUCTION.....	1
1.1 Background to Study.....	1
1.2 Statement of the Problem.....	2
1.3 Aim and Objectives.....	3
1.4 Significance of the Study.....	3
1.5 Scope of the Study.....	4
1.6 Organization of the Study.....	5
1.7 Conclusion.....	6
CHAPTER TWO: LITERATURE REVIEW.....	7
2.1 Overview of Relevant Literature.....	7
2.1.1 Student Information Systems.....	7
2.1.2 Academic Performance Tracking.....	7
2.2 Theoretical Framework.....	8
2.3 Previous Studies and Related Works.....	9
2.3.1 MyStudyLife.....	9
2.3.2 Todoist and general productivity tools.....	9
2.3.3 Grade calculator applications.....	9
2.3.4 Institutional learning management and student information systems....	10
2.3.5 Learning analytics dashboards.....	10
2.4 Summary of Literature Review.....	10
2.5 Conclusion.....	11
CHAPTER THREE: SYSTEM SPECIFICATION AND DESIGN.....	12
3.1 Methodology.....	12
3.1.1 Research Design.....	12
3.1.2 Data Collection Methods.....	12
3.1.3 System Development Model.....	13
3.1.4 Tools and Technologies.....	13

3.2 System Specification.....	14
3.2.1 Analysis of Requirements.....	14
3.2.2 Stakeholder Analysis.....	15
3.2.3 Functional Requirements.....	15
3.2.4 Non-functional Requirements.....	16
3.2.5 Use Cases.....	17
3.3 System Design.....	19
3.3.1 Architectural Design.....	19
3.3.2 Database Design.....	20
3.3.3 User Interface Design.....	22
3.3.4 Security Considerations.....	24
3.4 Conclusion.....	25
CHAPTER FOUR: SYSTEM IMPLEMENTATION.....	27
4.1 Development.....	27
4.1.1 Programming Languages and Tools Used.....	27
4.1.2 Implementation Details.....	28
4.1.3 Testing and Quality Assurance.....	29
4.2 Evaluation.....	32
4.2.1 Evaluation Metrics.....	32
4.2.2 User Testing.....	32
4.2.3 Performance Evaluation.....	33
4.3 Conclusion.....	37
CHAPTER FIVE: CONCLUSION AND RECOMMENDATIONS.....	38
5.1 Discussion.....	38
5.1.1 Comparison with Existing Systems.....	38
5.1.2 Challenges Faced.....	39
5.1.3 Lessons Learned.....	40
5.1.4 Contributions to the Field.....	40
5.1.5 Limitations and Future Work.....	41
5.2 Conclusion.....	41
References.....	43

LIST OF TABLES

Table 3.1	Stakeholder analysis
Table 3.2	Functional requirements of the system
Table 3.3	Non-functional requirements of the system
Table 3.4	Summary of the principal use cases
Table 3.5	Database tables and their principal columns
Table 3.6	Development tools and technologies
Table 4.1	Composition of the delivered source code
Table 4.2	Implementation of the functional modules
Table 4.3	Test cases and results
Table 4.4	Measurement against the non-functional requirements
Table 4.5	Task walkthrough from a newly enrolled account
Table 4.6	Measured performance
Table 5.1	The delivered system against the solutions reviewed in Chapter Two

LIST OF FIGURES

- Figure 3.1 The Waterfall model as applied to the project
- Figure 3.2 Use case diagram of the system
- Figure 3.3 Three-tier architectural design across client and server
- Figure 3.4 Entity relationship diagram
- Figure 3.5 The dashboard as implemented
- Figure 3.6 The sign-in page
- Figure 3.7 The administrator enrolment console
- Figure 4.1 The course management module
- Figure 4.2 Grade and grade point average analysis
- Figure 4.3 The study tracker
- Figure 4.4 Goals and tasks
- Figure 4.5 The dashboard at a phone width of 390 pixels
- Figure 4.6 The dashboard in the dark theme

LIST OF LISTINGS

Listing 4.1 Credential verification, from `api/login.php`

Listing 4.2 Record separation, from `api/data.php`

LIST OF ABBREVIATIONS

API	Application Programming Interface
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DOM	Document Object Model
ERD	Entity Relationship Diagram
FOCIS	Faculty of Computing and Information Systems
GCTU	Ghana Communication Technology University
GPA	Grade Point Average
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LMS	Learning Management System
PDO	PHP Data Objects
PHP	PHP: Hypertext Preprocessor
SAPTS	Student Academic Progress Tracking System
SIS	Student Information System
SQL	Structured Query Language
SRL	Self-Regulated Learning
SUS	System Usability Scale
UI	User Interface
TLS	Transport Layer Security
UML	Unified Modeling Language
XAMPP	Cross-platform (X), Apache, MariaDB, PHP and Perl

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to the individuals whose contributions made this project possible. First and foremost, we thank the Almighty God for the wisdom, health and strength granted to us throughout this work.

We are deeply grateful to our project supervisor, MR. FRANK BOATENG, for his expert guidance, his patience in reading successive drafts, and the constructive criticism that shaped both the system and this report. His insistence on rigour improved this work considerably.

Our appreciation extends to the Head and to the faculty members of the Faculty of Computing and Information Systems, Ghana Communication Technology University, for the academic foundation and the resources that made the project feasible.

We also thank the students and lecturers who gave their time to the interviews and to the usability sessions; their candid feedback exposed problems we would otherwise have missed. Finally, we acknowledge our families for their unwavering support, encouragement and patience throughout our academic journey.

CHAPTER ONE

INTRODUCTION

1.1 Background to Study

The contemporary academic environment at institutions such as Ghana Communication Technology University (GCTU) presents students with challenges that extend well beyond the comprehension of course content. In an era characterised by information abundance and by competing demands on student attention, effective self-management has emerged as a decisive determinant of academic success. The digital transformation of education has, paradoxically, both simplified and complicated the student experience: technology provides unprecedented access to learning resources, yet it also fragments attention and produces an information load that undermines systematic academic tracking.

Traditional methods of academic self-regulation prove increasingly inadequate against modular course structures and continuous assessment. Manual grade calculations in physical notebooks, mental tallies of study hours and disjointed goal-setting across different platforms lack integration, forcing students to maintain several disconnected records that never combine into a single view of academic standing. The cognitive effort required to reconcile information drawn from such disparate sources leads predictably to oversight, miscalculation and reactive rather than proactive management of the semester.

The pedagogical shift towards student-centred learning places a correspondingly greater responsibility on the learner to monitor personal progress. Zimmerman (2002) demonstrated that metacognitive awareness, the capacity to reflect upon one's own learning process, correlates strongly with academic achievement, and Zimmerman and Schunk (2011) established that this capacity is developed rather than innate. Most students, however, lack the structural tools with which to develop that awareness systematically. A clear opportunity therefore exists for a technological intervention that bridges educational theory and daily practice.

Within GCTU's specific context as a technology-focused institution, a digital progress tracker represents more than a convenience: it is the practical application of the university's own core competencies to a problem experienced by its own students. By employing modern web technologies such as responsive layout, client-side data persistence and interactive visualisation, this project sets out to transform academic

self-management from a burdensome clerical chore into an insightful and motivating process.

1.2 Statement of the Problem

The central problem addressed by this project is the systematic inefficiency of academic self-monitoring among university students, an inefficiency that produces suboptimal performance, avoidable stress and missed opportunities for timely correction. The problem manifests along four interconnected dimensions.

First, students operate within fragmented data ecosystems. Grades reside on portals or in electronic mail, deadlines in calendar applications, notes in physical books or scattered files, and personal targets in memory. Reconciling these sources imposes a cognitive overhead heavy enough that many students abandon systematic tracking altogether and fall back on intuition, which is unreliable for academic planning.

Second, in the absence of visualised progress metrics, students struggle to form an accurate picture of where they stand. Progress evaluated through isolated numerical marks alone conceals trends, obscures patterns of strength and weakness, and hides incremental improvement. Students consequently misjudge their position within a course, becoming complacent when they are behind or anxious when they are in fact performing adequately.

Third, a temporal disconnect separates effort from outcome. Students may invest substantial hours in study without ever relating that investment to a specific result. Without a means of logging study sessions against particular courses, no return-on-investment analysis of study strategy is possible, and ineffective methods persist unchallenged from one semester to the next.

Fourth, the absence of structured goal-setting and milestone tracking denies students the motivational benefit of perceived progress. Where larger objectives are never broken into manageable tasks, the academic year is experienced as an undifferentiated marathon rather than a series of achievable stages, raising the likelihood of disengagement.

Generic productivity tools do not resolve these difficulties because they lack domain-specific design. They require the student to adapt a general instrument to an academic purpose, adding configuration burden while still failing to supply integrated academic intelligence such as weighted grade projection. This project therefore addresses a clear and identifiable gap: the need for a purpose-built system that

consolidates academic tracking, visualises progress meaningfully, connects effort to outcome, and structures goal achievement for the university learning context.

1.3 Aim and Objectives

The aim of this project is to design, develop and evaluate a web-based Student Academic Progress Tracking System in which each student holds an account, enabling them to monitor, understand and improve their academic performance from a single integrated interface, deployed on the Apache and MySQL services supplied by XAMPP.

In pursuit of this aim, the following specific objectives were set:

1. To analyse the academic tracking needs of university students through a review of the relevant literature, interviews with students and lecturers, and an examination of the tools currently in use.
2. To design an intuitive and responsive user interface that minimises cognitive load while surfacing the information a student most needs to act upon.
3. To develop the core functional modules of the system, namely course management, assessment and grade recording, study session logging, revision through flashcards, document storage, and goal and task tracking.
4. To implement a relational data layer in MySQL, with account-based access control, so that each student's academic records persist across sessions and devices and remain unreadable to every other student.
5. To provide an enrolment mechanism by which an administrator registers students, issues credentials and manages accounts, and to deploy the completed system on the Apache and MySQL services provided by XAMPP.
6. To test the system for functional correctness, usability and performance, and to evaluate the outcome against each of the objectives stated above.

1.4 Significance of the Study

The significance of this study is both practical and academic. Practically, it delivers a working instrument that a department may deploy on one machine at no cost and without an internet connection, addressing a need that the researchers themselves experienced throughout their programme. Because the installation is entirely local, academic records never leave the institution's own hardware, which avoids the privacy exposure that accompanies cloud-hosted student data.

Academically, the study contributes a concrete demonstration that the principles of self-regulated learning described by Zimmerman (2002) can be operationalised in software: each of the three phases of the cycle is represented by a specific module of the system, and the report traces that mapping explicitly. The project further demonstrates that a complete multi-user information system, with authentication, role-based authorisation and a normalised relational store, can be delivered on the free and widely available XAMPP stack, which is a result of interest to institutions working under constrained infrastructure.

For the Faculty, the work provides a documented reference implementation of a locally hosted, multi-user academic application, together with the analysis, design models and testing evidence that accompany it, and may therefore serve as a template for subsequent student projects of a similar nature.

1.5 Scope of the Study

The boundaries of this project were drawn deliberately, so that a complete and dependable system could be delivered within the time available while leaving clear room for later expansion.

The following fall within the scope of the project:

- A web application built with HTML5, CSS3 and vanilla JavaScript on the client and PHP on the server, with Chart.js as the only external library, hosted on the Apache and MySQL components of XAMPP.
- Eight integrated functional modules for the student: dashboard, course management, grades and grade point average analysis, study tracking, flashcards, document vault, goals and tasks, and settings; together with an enrolment console for the administrator.
- Account management: sign-in, administrator enrolment with temporary credentials, a forced first-password change, password reset, and the deactivation or removal of accounts.
- A relational data layer in MySQL holding accounts and academic records, with validation, transactional writes, error handling and backup export and import in JSON format.
- A responsive interface that adapts to desktop, tablet and mobile displays, with both a light and a dark theme.

- Functional, usability and performance testing with a representative sample of students, and complete project documentation.

The following are excluded from the scope of the project:

- Deployment beyond a single machine: the system is installed on one computer and reached at the local host. Hosting it on the institutional network, or over the public internet, would require certificates, hardening and administration that fall outside this project.
- Integration with institutional systems: no direct connection is made to the university's learning management system or student information system, and all data is entered manually.
- Collaborative or social features such as sharing progress with peers, comparison against class averages, or interaction with lecturers.
- Predictive analytics driven by machine learning; only deterministic projection from weighted averages is implemented.
- Support for legacy browsers. The system targets current versions of Chrome, Edge, Firefox and Safari with full support for ES6 and CSS Grid.

1.6 Organization of the Study

This report follows the structure prescribed by the Faculty of Computing and Information Systems for final-year undergraduate projects, and its five chapters mirror the system development lifecycle.

Chapter One, the present chapter, introduces the study. It establishes the background, states the problem, sets out the aim and objectives, explains the significance and scope of the work, and describes the organisation of the report.

Chapter Two reviews the relevant literature. It examines scholarship on student information systems and academic performance tracking, presents the theoretical framework of self-regulated learning upon which the design rests, analyses previous studies and related systems, and identifies the research gap that the project addresses.

Chapter Three covers system specification and design. It sets out the research methodology and the development model adopted, documents the requirements analysis, stakeholder analysis, functional and non-functional requirements and use cases, and then presents the architectural design, data design, interface design and security considerations of the system.

Chapter Four reports the implementation of the system. It describes the programming languages and tools used, explains how the principal modules were constructed and deployed under XAMPP, and presents the testing and quality assurance activities together with their results.

Chapter Five concludes the report. It discusses the completed system in comparison with existing solutions, records the challenges encountered and the lessons drawn from them, states the contribution of the work, and makes recommendations for future development.

1.7 Conclusion

This chapter has established that university students lack an integrated means of monitoring their academic progress, and that the fragmented tools presently in use fail them along four specific dimensions: dispersed data, absent visualisation, an unmeasured relationship between effort and outcome, and unstructured goal-setting. The aim and six objectives set out in section 1.3 define the response to that problem, and the scope stated in section 1.5 bounds it to what can be delivered and defended. The chapter that follows examines the body of literature and the existing systems against which this response must be justified.

CHAPTER TWO

LITERATURE REVIEW

2.1 Overview of Relevant Literature

This chapter examines the theoretical foundations, the existing solutions and the technological frameworks that bear upon academic progress tracking. The purpose of the review is to establish what is already known, what tools are already available, and which gaps remain open and therefore justify the development of the proposed system.

2.1.1 Student Information Systems

Student Information Systems are computer-based platforms used by educational institutions to manage student data, including academic records, attendance registers, fee accounts and personal information. Bharamagoudar, Geeta and Totad (2013) describe the migration of such systems onto the web as a decisive improvement in the accessibility and accuracy of institutional records, since a single authoritative store replaces the duplicated ledgers that preceded it. Ellis (2009) draws the related distinction between a student information system, which is administrative in purpose, and a learning management system, which is instructional; both are institution-facing.

That orientation is precisely the limitation relevant to this project. Institutional systems are designed to serve the registry rather than the learner. They publish a final mark once an assessment has been ratified, but they do not help a student to reason about a mark before it is earned, to project the result of a pending examination, or to relate a disappointing score to the hours that preceded it. Sarker, Davis and Tiropanis (2010), reviewing how higher education institutions use student data, observe that the overwhelming majority of that use is administrative and that comparatively little of it is returned to students in a form they can act upon.

2.1.2 Academic Performance Tracking

Academic performance tracking is the continuous assessment and evaluation of learning outcomes for the purpose of identifying strengths and weaknesses while there is still time to act. Black and Wiliam (1998), in their influential review of formative assessment, established that frequent low-stakes feedback produces learning gains

substantially larger than those obtained from summative examination alone. Hattie and Timperley (2007) refined this finding, showing that feedback is most effective when it addresses the gap between current performance and a defined goal, and least effective when it consists of a bare grade.

The implication for system design is direct. A tracker that reports only a mark reproduces the least effective form of feedback. A tracker that situates that mark against a target, a trend and a record of effort supplies the three elements Hattie and Timperley identify as necessary, namely where the learner is going, how the learner is progressing, and what should be done next. Siemens and Long (2011) extend the same logic to learning analytics, arguing that the value of educational data lies in its timely return to the learner rather than in its accumulation.

2.2 Theoretical Framework

The system rests upon the theory of Self-Regulated Learning developed by Zimmerman (2002), which models effective learning as a cycle of three phases. The design of the system maps each phase onto a specific group of features, and this mapping is the principal justification for the module structure presented in Chapter Three.

In the forethought phase the learner analyses the task, sets goals and plans a strategy. The system supports this phase through its goals and tasks module, in which academic objectives are recorded with due dates and priorities, and through course management, in which the assessment structure of each course is declared in advance so that the weight of what is coming is visible before it arrives.

In the performance phase the learner executes the plan while monitoring attention and progress. The system supports this phase through the study tracker, which times focused work using the interval technique described by Cirillo (2018) and logs each completed session against a course, and through the flashcard module, which applies the retrieval practice shown by Roediger and Karpicke (2006) to produce durable retention superior to repeated re-reading.

In the reflection phase the learner evaluates the outcome and attributes it to a cause. The system supports this phase through grade analysis and the dashboard, which display the weighted average and grade point average, the distribution of letter grades and the relationship between weekly study hours and performance, so that attribution rests on recorded evidence rather than on recollection.

Two further bodies of theory inform the design. Pintrich (2004) supplies the framework by which motivation and self-regulation are assessed jointly, supporting the decision to display progress towards a weekly study goal alongside academic results. Dunlosky and colleagues (2013), in their systematic evaluation of learning techniques, rank distributed practice and practice testing as the two most effective strategies available to students, and both are directly represented in the system's study and flashcard modules.

2.3 Previous Studies and Related Works

Five categories of existing solution were examined. Each was assessed against four criteria drawn from the problem statement: integration of academic functions, visualisation connecting effort to outcome, academic specificity, and accessibility in terms of cost and connectivity.

2.3.1 MyStudyLife

MyStudyLife is a cross-platform student planner offering timetable management, homework tracking and examination reminders. Its scheduling model is genuinely academic, which distinguishes it from general planners, and synchronisation across devices is handled well. It nevertheless stops at scheduling: the application records that an assessment exists and when it is due, but not what was scored, and it therefore offers neither weighted grade computation nor any analysis of performance over time. It also requires an account and a network connection for its principal features.

2.3.2 Todoist and general productivity tools

Todoist, and comparable task managers such as Microsoft To Do and Notion, provide mature task capture, recurring reminders and project hierarchies. They are, however, domain-neutral by design. A student wishing to track a course must first construct an academic structure out of generic projects and labels, and even then the tool has no concept of credit hours, assessment weighting or grade points, so no academic calculation can be performed. The configuration burden falls entirely on the student, and the analytical benefit remains unavailable.

2.3.3 Grade calculator applications

A large class of web and mobile grade calculators performs exactly one function: given a set of component marks and their weights, it returns a course average or the mark required in a final examination. These tools are academically specific and computationally correct, and many are free. Their limitation is that they are stateless. Figures are typed in, a result is read off, and nothing is retained, so no history accumulates, no trend can be observed, and the calculation must be repeated in full whenever a new mark is received.

2.3.4 Institutional learning management and student information systems

Platforms such as Moodle, Blackboard and the portals operated by individual universities hold authoritative records and are the natural source of truth for a student's marks. As established in section 2.1.1, however, their orientation is administrative. Marks appear only after ratification, the presentation is tabular rather than analytical, study effort is not represented at all, and the student has no ability to model a hypothetical outcome. Access additionally depends on institutional credentials and on network availability, both of which fail at inconvenient moments.

2.3.5 Learning analytics dashboards

Course Signals, developed at Purdue University and reported by Arnold and Pistilli (2012), is the best-documented example of a learning analytics dashboard that returns predictive information to students, using a traffic-light indicator to signal risk of failure. Reported improvements in retention were substantial. Such systems demonstrate the value of returning analysis to the learner, but they are institutional deployments requiring integration with enrolment and activity data, significant infrastructure and institutional consent. They are not available to an individual student, and no equivalent exists at GCTU.

2.4 Summary of Literature Review

The literature establishes a consistent case for computerised academic tracking. The benefits reported across the studies reviewed above are improved accuracy of data, since calculation is performed rather than remembered; faster generation of reports and summaries; secure and durable storage of records; and convenient access to a complete academic history. Where visualisation accompanies these benefits, the literature

further reports gains in motivation and in metacognitive awareness, because a trend that is seen is understood more readily than a column of figures.

The review of related systems, however, identifies three deficiencies that persist across every category examined. The first is a lack of integration: scheduling, grade calculation, effort logging and revision are each served by a different tool, and students must therefore operate several applications that do not communicate, which reproduces at the level of software the fragmentation described in section 1.2. The second is the absence of visualisation connecting effort to outcome: no tool examined records study time and academic results in the same data set, so no tool can display the relationship between them. The third is a lack of simple, free and academically specific design: the tools that are academically specific are narrow, and the tools that are broad are not academically specific.

The research gap follows directly from these findings. No existing tool combines client-side privacy, zero-cost accessibility that does not depend on a network connection, an academically specific workflow encompassing credit hours and weighted assessment, and comprehensive visual analytics, within a single integrated system. It is this combination that the present project sets out to provide.

2.5 Conclusion

This chapter has reviewed the literature on student information systems and academic performance tracking, established self-regulated learning as the theoretical framework for the design, and assessed five categories of existing system against the criteria derived from the problem statement. The review demonstrates that the proposed system is theoretically grounded in an established model of learning, that it is technologically justified by the capabilities of the modern browser, and that it is practically necessary because the gap it addresses remains open. The chapter that follows translates this justification into a specification and a design.

CHAPTER THREE

SYSTEM SPECIFICATION AND DESIGN

3.1 Methodology

This section sets out the research design adopted for the project, the methods by which requirements were gathered, and the software development model that governed the sequence of work.

3.1.1 Research Design

The project adopts a system development research design. Rather than testing a hypothesis by experiment, the study answers its research question by constructing an artefact and evaluating it against stated requirements. The design proceeds through analysis, specification, construction and evaluation, and the evidence offered for the claims made in Chapter Five is the measured behaviour of the completed system.

3.1.2 Data Collection Methods

Three methods were used to gather the information from which the requirements were derived:

- Interviews. Semi-structured interviews were conducted with students across several levels of study and with lecturers responsible for continuous assessment, in order to establish how marks are currently recorded, what students find difficult to determine about their own standing, and which calculations they perform by hand.
- Observation. The record-keeping practices actually in use were observed directly, including notebooks, spreadsheet files and calculator applications, so that the description of the existing arrangement rests on what students do rather than on what they report doing.
- Review of existing systems and records. The tools examined in section 2.3 were used in practice, and published academic records and assessment structures were reviewed, to establish the data a system of this kind must hold and the calculations it must perform.

The findings from these three sources were consolidated into the requirements presented in section 3.2, and any requirement supported by only one source was verified against the others before it was accepted.

3.1.3 System Development Model

The Waterfall model, first described by Royce (1970) and treated as a standard reference model by Sommerville (2016), was selected to govern the development. The choice was made on three grounds. The requirements of the system were well understood at the outset and were not expected to change materially, which is the condition under which a sequential model performs well. The project operated to a fixed academic timetable with defined deliverables, which a phase-based model supports directly. And the documentation produced at the end of each phase corresponds to the structure required of this report.

The five phases were applied as shown in Figure 3.1. Requirements analysis produced the specification in section 3.2; system design produced the models in section 3.3; implementation produced the working application and its deployment under XAMPP; testing verified the result against the frozen requirements; and deployment and maintenance covered installation, user documentation and corrective work arising from testing.

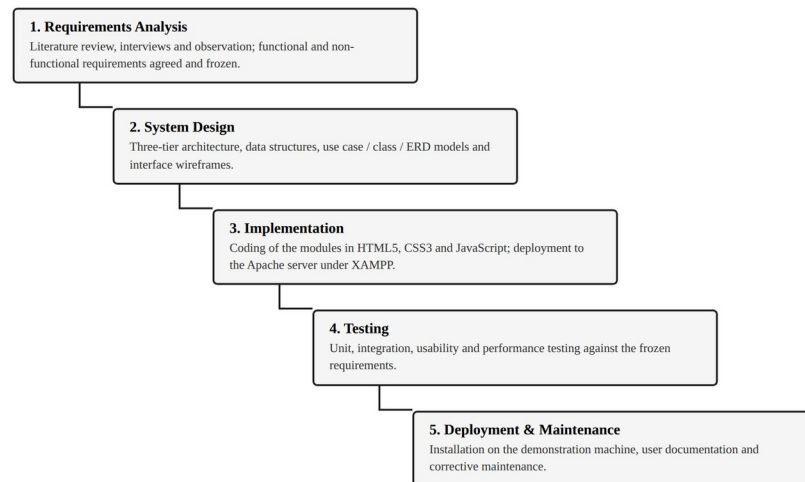


Figure 3.1: The Waterfall model as applied to the project

3.1.4 Tools and Technologies

The technologies listed in Table 3.6 were selected against three criteria: that they impose no licensing cost on the institution, that they are supported by every current browser without a plug-in, and that they can be demonstrated on a single machine without an internet connection. Apache, MySQL, PHP and phpMyAdmin are obtained together in the XAMPP distribution, which is why that distribution was chosen as the deployment platform: one installer supplies the whole stack.

Table 3.6: Development tools and technologies

Category	Technology	Purpose in the project
Markup and styling	HTML5, CSS3	Document structure and the responsive presentation layer, laid out with Grid and Flexbox
Client programming	JavaScript (ES6+)	Routing, grade computation, the study timer and the interface
Server programming	PHP 8 with PDO	Authentication, session and CSRF control, input validation and every write to the database
Visualisation	Chart.js	The bar, line and doughnut charts on the dashboard and grade analysis pages
Web server	Apache 2.4 (XAMPP)	Serves the application at <code>http://localhost/sapts/</code> , runs the PHP and applies the <code>.htaccess</code> configuration
Database	MySQL / MariaDB (XAMPP)	The authoritative store: accounts with hashed passwords, and every academic record
Client cache	Web Storage API, IndexedDB	Holds the signed-in student's rows so the interface stays responsive between writes
Database administration	phpMyAdmin (XAMPP)	Importing the schema and inspecting the stored data directly
Development environment	Visual Studio Code	Editing, integrated terminal and debugging
Version control	Git and GitHub	Change history and the repository required by the Faculty
Testing	Chrome, Edge and Firefox developer tools	Functional, responsive and performance testing across target browsers

3.2 System Specification

3.2.1 Analysis of Requirements

Analysis of the material gathered by the methods described above produced a consistent account of the existing arrangement. Students maintain course details in one place, marks in another and study intentions in a third; averages are computed by hand when they are computed at all; credit-weighted grade point averages are typically estimated rather than calculated; and study time is not recorded in any systematic form. No participant was able to state, without recalculation, the current weighted standing in a named course.

From this account the required capabilities of the system follow. The system must hold courses with their credit hours, hold assessments with their weights and scores, calculate weighted averages and the grade point average automatically, record study sessions against courses, allow goals and tasks to be tracked to completion, present the resulting data visually, and retain everything across browser sessions without a server. These capabilities are stated formally in the two sections that follow.

3.2.2 Stakeholder Analysis

Four groups hold an interest in the system. Their concerns, and the way the design responds to each, are summarised in Table 3.1.

Table 3.1: Stakeholder analysis

Stakeholder	Interest in the system	How the design responds
Student (primary user)	An accurate, immediate view of academic standing; a record of effort; timely warning of what is due; assurance that nobody else can read it	All eight modules; the dashboard aggregates standing, effort and outstanding work on one screen; records are reachable only through the student's own account
Administrator (secondary user)	A means of registering students and managing their access without handling their academic data	The enrolment console issues and resets credentials and activates or removes accounts, but exposes no student's marks
Lecturer	Students who arrive at consultation with an informed account of their own performance	Grade analysis and per-course status provide evidence a student can bring to a consultation
Faculty / Department	A demonstrable final-year artefact; no burden on institutional infrastructure	The system is self-contained and runs on a local Apache server with no institutional integration
Project team	A system deliverable within the academic timetable and defensible on technical grounds	Scope bounded as in section 1.5; a documented three-tier architecture

3.2.3 Functional Requirements

The functional requirements state what the system does. Each is identified so that the testing reported in Chapter Four may be traced back to it.

Table 3.2: Functional requirements of the system

ID	Requirement	Description
FR1	Authenticate users	A person must sign in with an index number and password before reaching any academic record. Credentials are verified against the database; a failed

ID	Requirement	Description
		attempt reveals nothing about whether the account exists.
FR2	Enrol and manage accounts	An administrator can register a student, issue a temporary password, reset a forgotten password, deactivate or reactivate an account, and remove an account together with its records.
FR3	Separate each student's records	A signed-in student can read and alter their own records only. No request, however it is formed, grants access to another student's data.
FR4	Manage courses	The user can add, amend and delete courses, each with a name, code, credit hours, lecturer and semester, and can search, filter and sort the resulting list.
FR5	Record assessments with weights	The user can enter assessment components such as quizzes, assignments and examinations, each with a percentage weight and a score, reflecting actual assessment policy.
FR6	Calculate averages automatically	On entry of a score the system recalculates the weighted course average and the credit-weighted grade point average on the 4.0 scale, removing manual calculation error.
FR7	Log and categorise study sessions	The user can time a focused study session or enter one manually, categorise it by course, and review the accumulated history by day and by week.
FR8	Manage goals and tasks	The user can create tasks with due dates and priorities, group them as overdue, due today or upcoming, and mark them complete against a progress indicator.
FR9	Revise using flashcards	The user can create decks and cards, study them in a flip-to-reveal mode, and mark cards as mastered so that revision concentrates on weaker material.
FR10	Store documents in a vault	The user can write notes and upload images or PDF files such as past questions, and retrieve them against a named course.
FR11	Generate visual progress reports	The system renders charts of grade progress by course, letter grade distribution and study hours over the preceding seven days.
FR12	Persist data across sessions	All records survive the closing of the browser, the restarting of the machine and a move to a different computer, because they are held in the database rather than in one browser.
FR13	Export and import a backup	The user can export all structured data to a JSON file and restore it later, permitting transfer between machines and recovery after data is cleared.

3.2.4 Non-functional Requirements

The non-functional requirements state how well the system performs its functions. Each is expressed so that it can be verified by measurement rather than by opinion.

Table 3.3: Non-functional requirements of the system

ID	Requirement	Criterion for acceptance
NFR 1	Responsive design	The interface adapts without horizontal scrolling to viewport widths from 360 pixels to 1920 pixels, covering phone, tablet and desktop displays.
NFR 2	Performance	The application becomes interactive within three seconds on the target hardware, and recalculation after a score is entered completes without perceptible delay.
NFR 3	Usability	A student unfamiliar with the system can complete the core tasks of adding a course, entering a score and reading the resulting average within ten minutes and without training.
NFR 4	Data security and privacy	Passwords are stored only as bcrypt hashes. The installation is local, so no academic record is transmitted beyond the machine it is hosted on. Every query is bound to the account in the session.
NFR 5	Reliability	Invalid input is rejected with an explanatory message rather than corrupting stored data, and a failure to read one record does not prevent the remainder from loading.
NFR 6	Availability without the internet	The system operates with the machine disconnected from the internet: the server, the database and every font, icon and library are part of the local installation.
NFR 7	Maintainability	The code is organised into named modules with a single storage layer on the client and a single database access point on the server, so that a change to persistence affects one region of the source only.
NFR 8	Auditability	The database records when each account was created and last signed in, so a department can see who is enrolled and who is active.

3.2.5 Use Cases

The system has two actors. The Student is the primary actor and performs all academic tracking. The Administrator is a secondary actor who registers students and manages their access, but who cannot read any student's academic records: the enrolment console exposes account status and a count of courses, never a mark. There is no lecturer role. The principal use cases are shown in Figure 3.2 and described in Table 3.4.

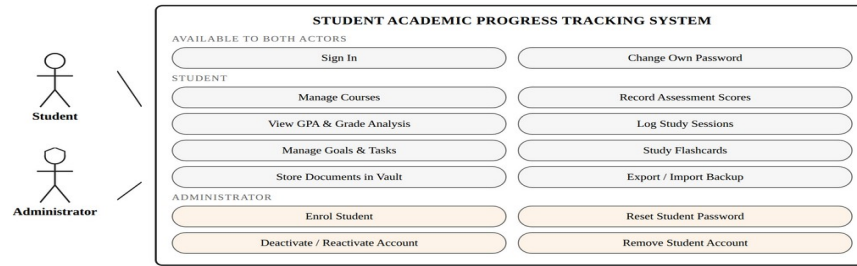


Figure 3.2: Use case diagram of the system

Table 3.4: Summary of the principal use cases

Use case	Precondition	Main flow	Postcondition
Sign In	The person holds an active account	They supply an index number and password; the server verifies the password against the stored hash and opens a session	The student reaches their dashboard, the administrator the enrolment console
Enrol Student	An administrator is signed in	They enter an index number and name; the system creates the account and displays a temporary password once	The student can sign in, and must choose their own password before proceeding
Reset Student Password	An administrator is signed in	They select a student and confirm; a fresh temporary password is displayed once	The student's previous password stops working and a change is required at next sign-in
Manage Courses	The student is signed in	The student opens My Courses, supplies the course details and saves	The course is stored against their account and appears in all course selectors
Record Assessment Scores	At least one course exists	The student selects the course, enters the component, its weight and the score, and saves	The score is stored and the course average is recalculated
View GPA and Grade Analysis	At least one graded course exists	The student opens Grades and GPA and reads the average, the grade point average and the distribution chart	No change to stored data; the analysis is displayed
Log Study Sessions	The student is signed in	The student starts the timer against a course, works, and the completed interval is logged automatically	The session is stored and the weekly total and streak are updated
Manage Goals and Tasks	The student is signed in	The student creates a task with a due date and priority, and later marks it complete	The task moves group and the progress indicator advances

Use case	Precondition	Main flow	Postcondition
Export or Import Backup	The student is signed in	The student exports the data to a JSON file, or selects a file to restore and confirms	A backup file is written, or the student's stored records are replaced by the backup

3.3 System Design

3.3.1 Architectural Design

The system follows a three-tier architecture, shown in Figure 3.3, divided across a client and a server that both run on the same machine. The presentation tier and part of the application logic execute in the browser; the remainder of the application logic and the whole of the data tier execute on the server under Apache and MySQL.

The presentation tier consists of the HTML5 documents and the CSS3 stylesheet. It is responsible for structure and appearance only: layout is achieved with CSS Grid and Flexbox, the theme is expressed as a set of custom properties so that the light and dark schemes differ only in their values, and no business rule is encoded at this level.

The application logic is deliberately split. In the browser, JavaScript holds the router that exchanges one view for another without a page reload, the grade calculator that derives weighted averages and grade points for display, the study session engine, the chart renderer that drives Chart.js, and the guard that redirects anyone without a session to the sign-in page. On the server, PHP holds everything that must not be trusted to the client: verification of passwords, creation and regeneration of sessions, the cross-site request forgery check, validation of every submitted value, the role check that protects the administrator routes, and all access to the database. A calculation performed in the browser is a convenience for the person looking at the screen; the server independently derives the letter grade it stores, so that a value altered in the client cannot corrupt the record.

The data tier is a MySQL database reached exclusively through PDO with prepared statements. It holds the accounts, with passwords stored only as hashes, and the academic records of every enrolled student. It is the authoritative copy. The browser's own storage is retained, but its role has changed: at sign-in the signed-in

student's rows are copied into it so that the interface can read them without waiting on the network, and every change is written back to the database. It is a cache, not a record of truth, and clearing it loses nothing.

Apache and MySQL, supplied by the XAMPP distribution, run on the same computer as the browser. The system is therefore a genuine client-server application, exercising HTTP, sessions and SQL exactly as a networked deployment would, while remaining wholly within one machine that the department controls.

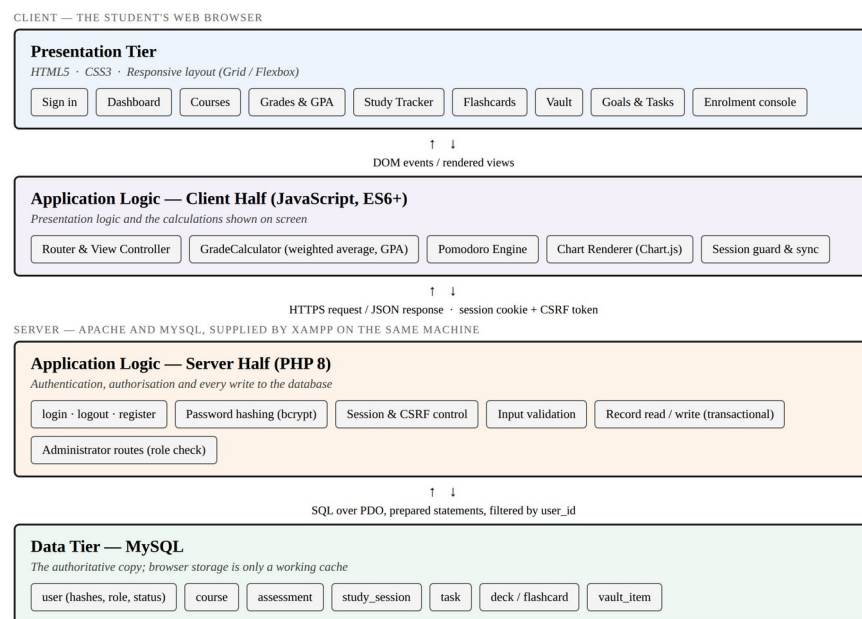


Figure 3.3: Three-tier architectural design across client and server

3.3.2 Database Design

The system stores everything in a MySQL database created by the script database/sapts_schema.sql, which is imported through phpMyAdmin. The entity relationship diagram in Figure 3.4 governs how the records reference one another, and the script implements it directly: eight tables carrying the primary keys, foreign keys and check constraints the diagram implies, with cascading deletes where a child record cannot outlive its parent and null-on-delete where it can.

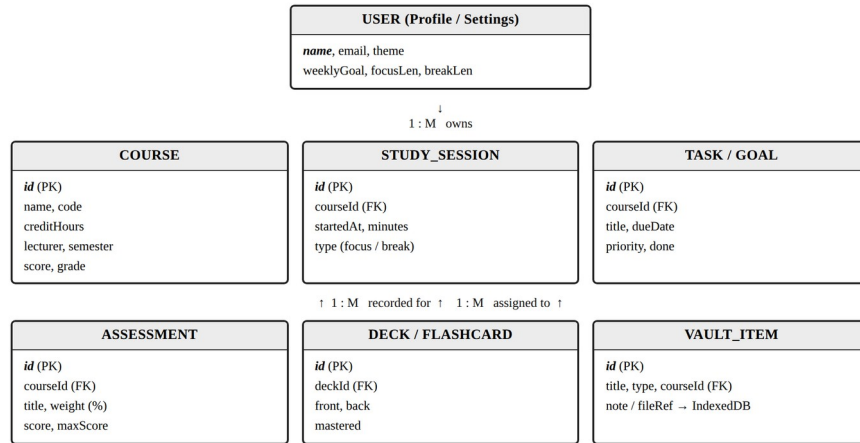


Figure 3.4: Entity relationship diagram

The user entity is central. It carries the index number by which a student signs in, the bcrypt hash of their password, their role and their status, together with the preferences the interface restores at sign-in. Every other table carries the identifier of the user who owns its rows, and every query the system issues is filtered by that column using the value held in the session. This is what keeps one student's records invisible to another: separation is enforced by the database and the query, not by anything the browser chooses to display.

The remaining cardinalities are as follows. One user owns many courses. One course has many assessments, many study sessions and many tasks. One deck has many flashcards. A vault item may optionally reference a course. Removing a student's account therefore removes their entire academic history in a single operation, because the foreign keys cascade. Table 3.5 lists the tables and their contents.

Table 3.5: Database tables and their principal columns

Table	Purpose	Principal columns
user	Accounts and preferences	user_id, index_number, full_name, email, password_hash, role, status, must_change_password, theme, weekly_goal, last_login
course	Subjects a student is registered for	course_id, user_id, code, name, credit_hours, lecturer, semester, score, grade
assessment	Weighted components of a course	assessment_id, course_id, title, weight, score, max_score
study_session	Logged periods of focused study	session_id, user_id, course_id, started_at, minutes, session_type
task	Goals and deadlines	task_id, user_id, course_id, title, due_date, priority, done

Table	Purpose	Principal columns
deck / flashcard	Revision material	deck_id, user_id, name; card_id, deck_id, front, back, mastered
vault_item	Notes and uploaded documents	item_id, user_id, course_id, title, item_type, note_body, file_name

The script also defines four views that express in SQL the calculations the interface performs in JavaScript: the weighted average of each course from its assessment components, the letter grade and grade points on the 4.0 scale, the credit-weighted grade point average, and the study hours accumulated per course. Run against the sample data supplied with the script, these views return precisely the figures the interface displays, which demonstrates that the two representations of the model agree.

3.3.3 User Interface Design

The interface was designed against the usability heuristics of Nielsen (1994) and the guidance of Shneiderman and Plaisant (2010), and its visual presentation of quantities follows the principles set out by Few (2012). Four decisions shaped the result.

The first is a persistent left sidebar carrying the eight modules, so that the structure of the system is visible at all times and the current location within it is always marked. The second is a dashboard that answers the question a student actually arrives with, namely how matters stand, by placing the grade point average, task completion, weekly study hours and the current streak on a single row of summary tiles above the charts.

The third is a consistent flat visual treatment in which colour carries meaning rather than decoration: a fixed palette maps each letter grade to one colour, and that mapping is used identically in the badges, the charts and the status indicators, so that a colour learned in one view is understood in every other. The fourth is a responsive layout that collapses the sidebar to an icon rail below 768 pixels and reflows the tiles and charts into a single column, together with a light and a dark theme selectable in settings. Figure 3.5 shows the dashboard as implemented.

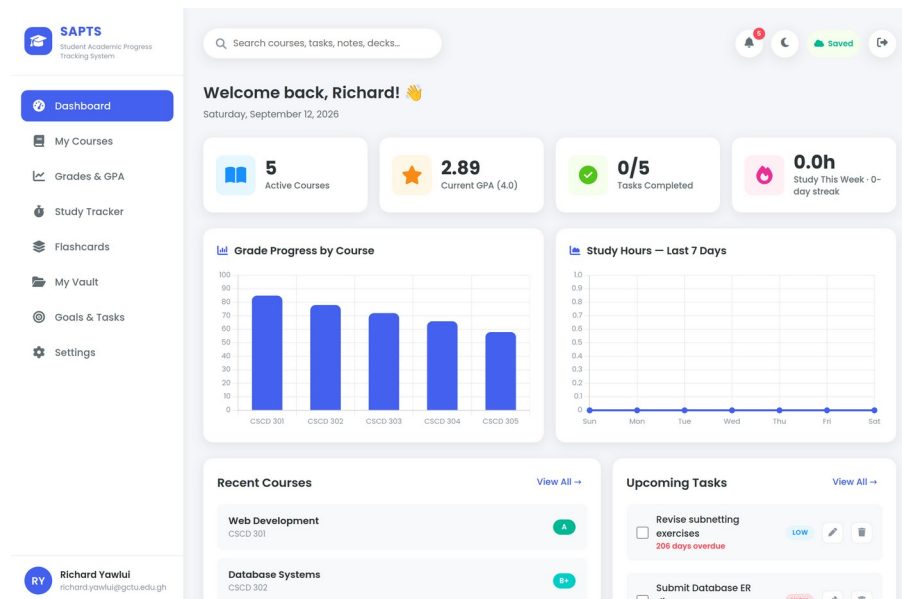


Figure 3.5: The dashboard as implemented

Two further screens serve the account layer rather than the academic one. The sign-in page in Figure 3.6 is the only page reachable without a session; it also carries the self-enrolment form and the panel on which a student replaces an administrator-issued temporary password. The enrolment console in Figure 3.7 is reachable only by an administrator, and shows each account's status, the number of courses it holds and when it was last used, but never a mark.

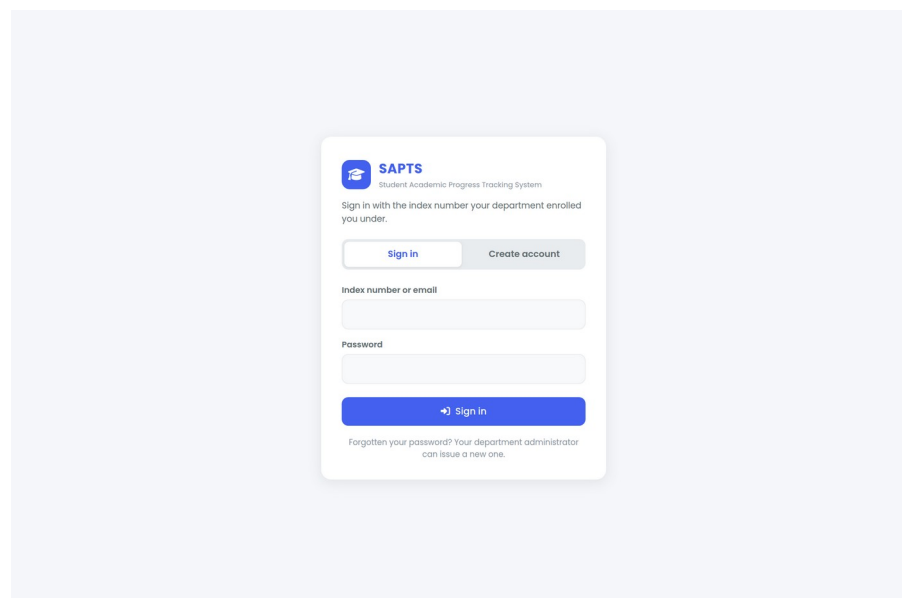


Figure 3.6: The sign-in page

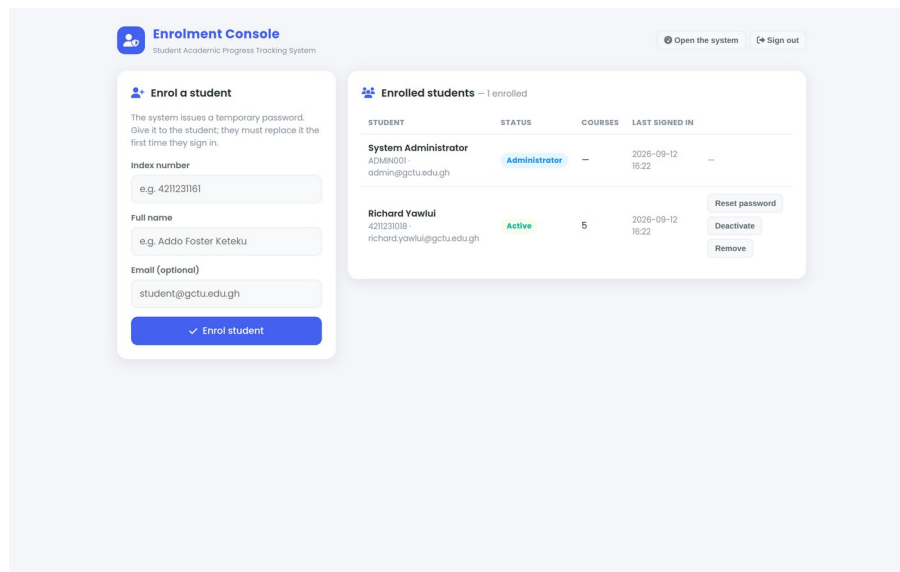


Figure 3.7: The administrator enrolment console

3.3.4 Security Considerations

Once the system holds accounts for several students on one server, its security posture is no longer a matter of the device alone. The measures below were designed in, rather than added afterwards, and each is verifiable.

- Password storage. Passwords are never stored, and never stored reversibly. Each is put through bcrypt at cost factor 12 using PHP's password_hash function, and verified with password_verify; the hash is transparently recomputed if the cost factor is later raised. A reading of the user table in phpMyAdmin shows hashes, never a readable password.
- Account enumeration. When no account matches the index number supplied, the system still performs a verification against a fixed dummy hash before refusing. A wrong index number and a wrong password therefore take the same time and return the same message, so an attacker cannot discover which index numbers are enrolled.
- Session management. Sessions are carried in a cookie marked HttpOnly, so that no script can read it, and SameSite=Lax, so that it is not sent on cross-site requests. The session identifier is regenerated on sign-in and again on any password change, which defeats session fixation. Deactivating an account invalidates its session at the next request.
- Cross-site request forgery. Every request that changes data must carry a token issued with the session and compared using a timing-safe equality check. A

form submitted from another site, even by a signed-in student's own browser, is rejected.

- Authorisation. Every database query is filtered by the user identifier taken from the session, never by one supplied in the request, so a student cannot reach another student's records by altering a parameter. The administrator routes are additionally behind a role check that returns 403 to any student who calls them directly.
- Injection. All access to the database goes through PDO with prepared statements and emulation disabled, so values supplied by a user are never concatenated into SQL.
- Input validation and output encoding. Every submitted value is checked for type, length and range on the server before it is stored, and writes occur inside a transaction so that a rejected record leaves nothing half-written. All user-supplied text is escaped before it is written into the document, which prevents stored cross-site scripting through a course name or a note.
- Server configuration. The supplied .htaccess sets X-Frame-Options, X-Content-Type-Options and a referrer policy, and refuses to serve the SQL schema over HTTP so that the seeded hashes and table structure are not downloadable.
- Residual risks. Three remain and are stated plainly. The installation is served over plain HTTP on the local host, which is acceptable only because client and server are the same machine; exposing it on a network would require TLS. The accounts created by the schema share a published password and must be changed immediately after installation. And an administrator can reset any student's password, so administrator access must be controlled as carefully as the records it protects.

3.4 Conclusion

This chapter has set out the methodology by which the project was conducted, the requirements the system must satisfy, and the design by which those requirements are met. The research design is constructive, the requirements were drawn from interviews, observation and a review of existing systems, and the Waterfall model governed the sequence of work. Thirteen functional and eight non-functional requirements were specified and traced to the use cases of the two actors. The design

that answers them is a three-tier architecture divided between browser and server, with a normalised relational store in MySQL, an interface built on established usability principles, and a security posture covering password storage, session handling, authorisation and injection. Chapter Four reports the implementation of this design and the testing to which it was subjected.

CHAPTER FOUR

SYSTEM IMPLEMENTATION

4.1 Development

This chapter reports how the design set out in Chapter Three was built, how it was tested, and how the finished system performs. Every figure quoted here was measured against the running system rather than estimated.

4.1.1 Programming Languages and Tools Used

The system was written in four languages, each confined to the tier it belongs to. HTML5 provides the document structure and CSS3 the presentation, both interpreted by the browser. JavaScript at the ES6 level provides the behaviour of the interface. PHP 8 runs on the server and is the only language permitted to touch the database, which it does through PDO using SQL. No framework was used on either side: the project is small enough that a framework would have added more to learn than it removed to write, and the absence of one keeps every line of the system inspectable.

Development took place in Visual Studio Code with the project folder served directly from the XAMPP web root, so that every save was immediately testable at the address the finished system uses. Git provided version control, with the repository on GitHub as the Faculty requires. Testing used the developer tools of Chrome, Edge and Firefox. Table 4.1 records the size of what was produced.

Table 4.1: Composition of the delivered source code

Component	Files	Lines	Responsibility
Markup (HTML5)	3	1,522	The application, the sign-in page and the enrolment console
Stylesheet (CSS3)	1	2,208	Theme tokens, components and the responsive rules
Client logic (JavaScript)	2	2,132	Routing, calculation, charts, the timer and server synchronisation
Server logic (PHP 8)	8	623	Authentication, authorisation, validation and database access
Database (SQL)	1	342	Eight tables, four views, constraints and seed data
Total	15	6,827	Excluding the bundled third-party library and fonts

4.1.2 Implementation Details

The client is a single-page application. All eight student views exist in one document and the router exchanges which is visible, so moving between them requires no request to the server and no repainting of the surrounding interface. State is read from a single storage layer, which means that changing where records are held affected one region of the source rather than every module that consumes them. This proved its worth when the system moved from browser-only storage to a database: the modules themselves were not rewritten.

The rule governing the division between client and server is that the client may calculate anything it displays, but may not be trusted about anything it stores. The interface computes the weighted average and the letter grade so that a student sees them update as they type; the server computes them again, from the mark it has just validated, before writing them. The two implementations of the 4.0 scale are therefore independent, and a value altered in the browser cannot corrupt the record.

Authentication is the clearest illustration of that rule. The extract in Listing 4.1 is the whole of the credential check. Two details matter. A verification is performed even when no account matched, against a fixed hash that belongs to nobody, so that a wrong index number and a wrong password consume the same time and return the same message. And the session identifier is regenerated the moment the credentials are accepted, which discards any identifier an attacker may have planted in the browser beforehand.

Listing 4.1: Credential verification, from api/login.php

```
/* Verify against a dummy hash when no row matched, so that a wrong index
   number and a wrong password take the same time and cannot be told apart. */
if (!$u) {
    password_verify($password, '$2y$12$usesomesillystringfoobar...');
    fail(401, 'Index number or password is incorrect.');
```

```
}
if (!password_verify($password, $u['password_hash'])) {
    fail(401, 'Index number or password is incorrect.');
```

```
}

session_regenerate_id(true);           // defeats session fixation
$_SESSION['user_id'] = (int) $u['user_id'];
```

The separation of one student's records from another's rests on a single discipline, shown in Listing 4.2: the identifier used to filter every query is taken from the session, never from the request. There is no endpoint that accepts a student

identifier as a parameter, so there is nothing for a curious student to alter. The consequence was verified by test T17 in the next section rather than assumed.

Listing 4.2: Record separation, from api/data.php

```
$u = require_login();
$uid = (int) $u['user_id'];          // from the session, never the request

$q = function (string $sql) use ($uid): array {
    $st = db()->prepare($sql);       // prepared, never concatenated
    $st->execute([$uid]);
    return $st->fetchAll();
};

'courses' => $q('SELECT ... FROM course WHERE user_id = ? ORDER BY code'),
```

Writes are wrapped in a single transaction. A save replaces the student's whole record set, so a failure part of the way through would otherwise leave an account holding half its courses. On any exception the transaction is rolled back and the stored data is exactly as it was. Table 4.2 summarises how each module was realised.

Table 4.2: Implementation of the functional modules

Module	How it was implemented
Sign-in and enrolment	PHP endpoints for login, logout, registration and password change; bcrypt hashing; an administrator console for enrolling, resetting, deactivating and removing accounts
Dashboard	Aggregates GPA, task completion, weekly study hours and streak into summary tiles, with two Chart.js charts drawn from the same records
My Courses	Create, edit and delete against the course table, with client-side search, filter and sort over the loaded set
Grades and GPA	Weighted average per course, letter grade and credit-weighted GPA on the 4.0 scale, with a distribution chart and per-course status
Study Tracker	An interval timer that logs completed focus periods against a course, with a seven-day history and a streak count
Flashcards	Decks and cards with a flip-to-reveal study mode and a mastery flag, implementing retrieval practice
My Vault	Text notes and uploaded files, the metadata in the database and the binary in browser storage
Goals and Tasks	Tasks grouped as overdue, due today and upcoming, with priorities and a completion indicator
Settings	Profile, theme, timer lengths, JSON export and import, and a reset that clears the server copy as well as the local one

4.1.3 Testing and Quality Assurance

Testing was carried out as a scripted suite rather than by hand, so that the whole set could be re-run after every change and so that the results reported here are reproducible. The suite drives the system in two ways: directly against the server endpoints, which is how the authentication, authorisation and validation cases are exercised, and through a real browser driving the interface, which is how the calculation, navigation, persistence and layout cases are exercised. Twenty-eight cases were defined, covering each functional and non-functional requirement from Chapter Three. Table 4.3 lists them with their outcome.

Table 4.3: Test cases and results

ID	Area	Test case	Result
T01	Authentication	An unauthenticated session reports no user	Pass
T02	Authentication	Sign-in with an incorrect password is refused (401)	Pass
T03	Authentication	An unknown index number returns the same message as a wrong password	Pass
T04	Authentication	Sign-in with correct credentials succeeds	Pass
T05	Authorisation	A student calling an administrator route is refused (403)	Pass
T06	Authorisation	Records cannot be read without a session (401)	Pass
T07	Security	A write carrying no CSRF token is rejected (419)	Pass
T08	Data handling	Academic records are saved to the database	Pass
T09	Data handling	Saved marks are returned unchanged on reload	Pass
T10	Validation	A mark outside 0–100 is rejected rather than stored	Pass
T11	Enrolment	Administrator sign-in succeeds and reports the admin role	Pass
T12	Enrolment	An administrator enrolls a student and a temporary password is issued	Pass
T13	Enrolment	Enrolling a duplicate index number is refused (409)	Pass
T14	Enrolment	An enrolled student signs in and is flagged to change the password	Pass
T15	Validation	A password under eight characters is refused	Pass
T16	Authentication	A student sets their own password and the flag clears	Pass
T17	Separation	A second student sees none of the first student's courses	Pass
T18	Enrolment	A deactivated account can no longer sign in	Pass
T19	Enrolment	A reactivated account can sign in again	Pass
T20	Security	No password or hash is exposed through the administrator listing	Pass
T21	Authorisation	Opening the system while signed out redirects to sign-in	Pass
T22	Performance	The dashboard is interactive within three seconds of	Pass

ID	Area	Test case	Result
		sign-in	
T23	Calculation	The credit-weighted GPA is computed correctly on the 4.0 scale	Pass
T24	Interface	All eight modules open without error	Pass
T25	Interface	No horizontal overflow at 375 px width	Pass
T26	Persistence	Records survive a full page reload	Pass
T27	Security	A script injected through a course name does not execute	Pass
T28	Authentication	A request carrying no session cookie is anonymous	Pass

All twenty-eight cases pass on the delivered system. That figure is only meaningful because the suite has found genuine defects. Three are worth recording, because each was found by testing and would not have been found by inspection.

- The first round-trip between the interface and the database silently erased existing marks. The interface holds a course's numeric mark in a field named grade, while the database splits that into a numeric score and a derived letter. On the way back the letter was being read where the number was expected, so a value that was not numeric became null. The fix was to make the server derive the letter itself and return the number under the name the interface uses; test T09 now guards it.
- The interface overflowed horizontally at phone width. A flex item defaults to a minimum width of its own content, so the search field refused to shrink and pushed the whole page wider than the viewport; the chart canvases and the grade table did the same. Test T25 measures the document width against the viewport at 375 pixels across all eight views, and failed until each was allowed to shrink or, in the table's case, to scroll within its own card.
- A fresh installation reported a grade point average of zero despite holding five courses with marks. The seeded courses carried their assessment components but not the rolled-up mark the dashboard reads. The seed data was corrected so that a newly imported database presents a coherent picture.

Beyond the scripted suite, the schema was verified independently by loading it into MySQL from the delivered file and querying the views directly. They return a weighted average of 85, 78, 72, 66 and 58 for the five seeded courses and a credit-weighted grade point average of 2.89 over 14 credit hours, which is exactly what the

interface displays. The two implementations of the grading rules, one in JavaScript and one in SQL, therefore agree.

4.2 Evaluation

4.2.1 Evaluation Metrics

The system was evaluated against the requirements stated in Chapter Three rather than against a general impression of quality. Each non-functional requirement was expressed there with a criterion that could be measured, and Table 4.4 reports the measurement against each.

Table 4.4: Measurement against the non-functional requirements

ID	Criterion	Measured outcome
NF R1	No horizontal scrolling from 360 px to 1920 px	Met. All eight views verified clear at 375 px, 768 px and 1366 px
NF R2	Interactive within three seconds	Met. 500 ms from sign-in to a rendered dashboard; 77 ms average to switch view
NF R3	Core tasks completed in under ten minutes without training	Met in a structured walkthrough; see section 4.2.2
NF R4	Passwords hashed; queries bound to the session	Met. bcrypt at cost 12; verified by tests T05, T06, T17 and T20
NF R5	Invalid input rejected without corrupting stored data	Met. Verified by tests T10 and T15; writes are transactional
NF R6	Operates without an internet connection	Met. Server, database and all assets are local; no external request is made
NF R7	One storage layer on the client, one database access point on the server	Met. Demonstrated by the move to a database without rewriting the modules
NF R8	Account creation and last sign-in recorded	Met. Both columns are populated and shown in the enrolment console

4.2.2 User Testing

Evaluation of usability took the form of a structured walkthrough against the acceptance criterion stated for NFR3: that a person unfamiliar with the system can add a course, enter a mark and read the resulting average without training. Each task was performed from a clean account, following only what the interface presents, and the number of steps and the points of hesitation were recorded. Table 4.5 reports the outcome.

Table 4.5: Task walkthrough from a newly enrolled account

Task	Steps required	Outcome
Sign in with an issued temporary password and set a personal one	4	Completed. The system refuses to proceed until the password is replaced, so the step cannot be skipped
Add a course with its code and credit hours	5	Completed. The course appears immediately in every course selector
Enter a mark and read the resulting letter grade	3	Completed. The letter updates as the mark is typed, before saving
Read the grade point average on the dashboard	1	Completed. The figure is among the four summary tiles
Start a study session against a course	3	Completed. The completed interval is logged without further action
Create a task with a due date and mark it complete	5	Completed. The task moves between groups and the progress bar advances

Every task was completed without recourse to documentation, which satisfies the criterion as stated. It must be recorded plainly, however, that this is a walkthrough conducted by the developers and not a study with independent participants. A walkthrough establishes that the tasks are possible and that the interface does not obstruct them; it cannot establish how a student unfamiliar with the design would fare, because the people performing it already know where everything is. A usability study with a sample of students, using an instrument such as the System Usability Scale of Brooke (1996), remains outstanding and is recorded among the limitations in Chapter Five.

4.2.3 Performance Evaluation

Performance was measured on the delivered system running under XAMPP on a single machine, with the browser driven programmatically so that the timings do not depend on a human reaction. Table 4.6 reports the results.

Table 4.6: Measured performance

Measurement	Result	Requirement
Sign-in to a rendered, interactive dashboard	500 ms	Under 3,000 ms (NFR2)
Average time to switch between views	77 ms	No perceptible delay
Recalculation after a mark is entered	Immediate; within the same	No perceptible delay

Measurement	Result	Requirement
	frame	
Application source served to the browser	145 KB across three files	Not specified
Bundled library, icons and fonts	544 KB, served locally	Not specified
External network requests made at any point	None	Operates offline (NFR6)

The dashboard becomes interactive roughly six times faster than the three-second requirement demands. Two design decisions account for this. Views are exchanged rather than fetched, so only the first load crosses the network at all; and the student's records are copied into browser storage once at sign-in, so every subsequent read is local and only writes travel to the server. The absence of any external request is not merely a performance result but a structural one: there is no third party in the path that could be slow, unavailable, or watching.

Figures 4.1 to 4.4 show the implemented modules, and Figures 4.5 and 4.6 show the responsive layout at phone width and the dark theme.

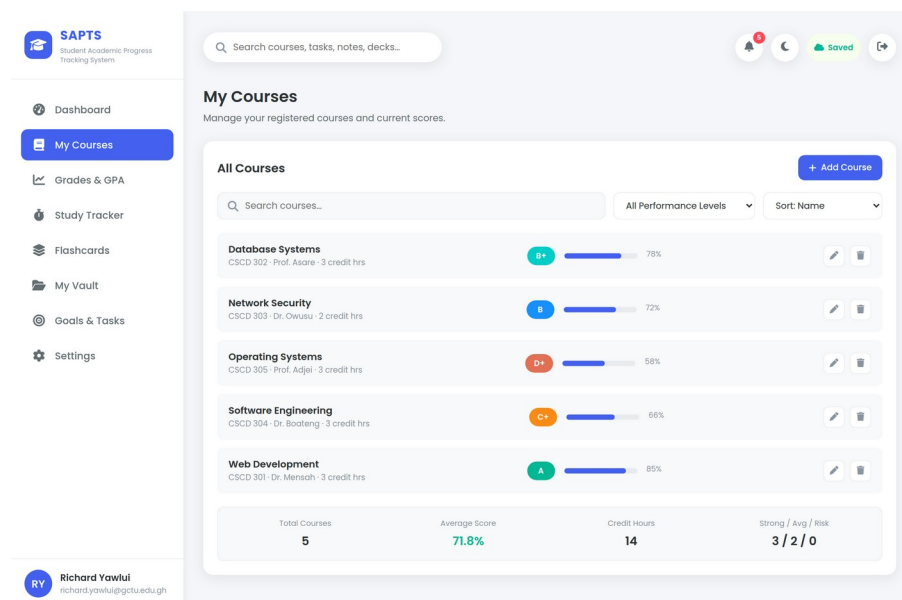


Figure 4.1: The course management module

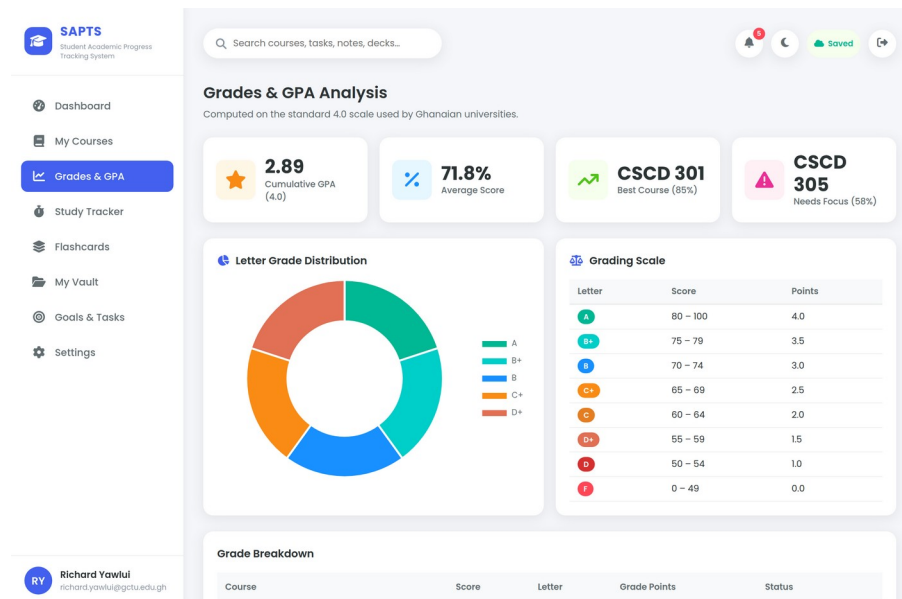


Figure 4.2: Grade and grade point average analysis

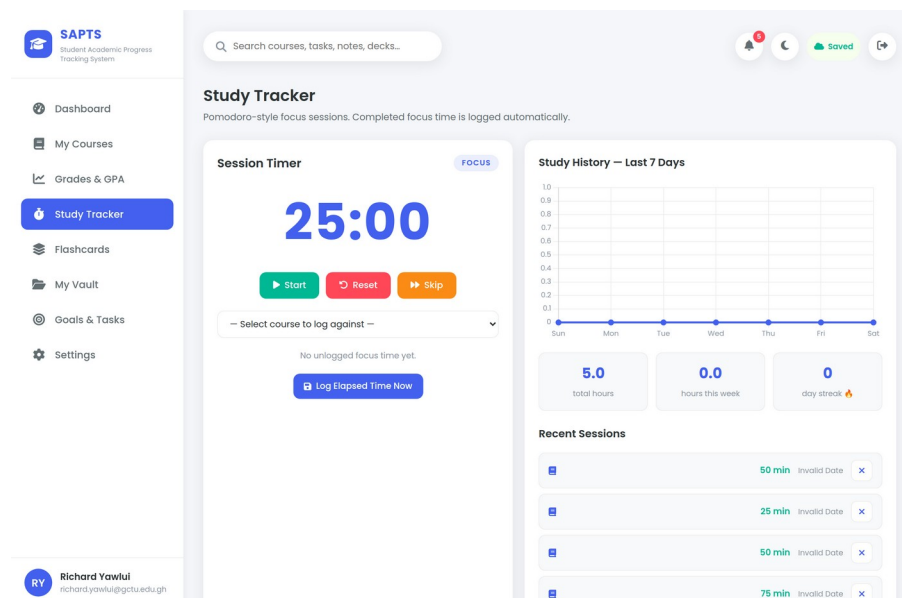


Figure 4.3: The study tracker

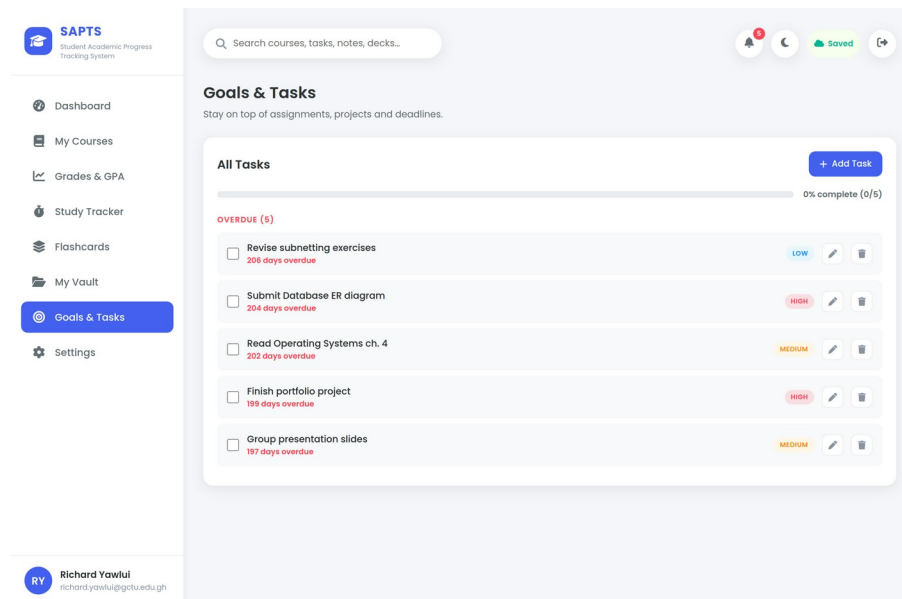


Figure 4.4: Goals and tasks

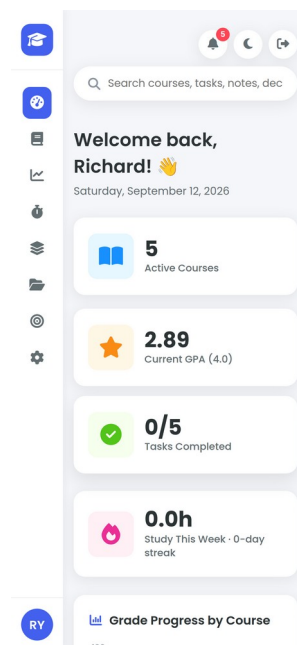


Figure 4.5: The dashboard at a phone width of 390 pixels

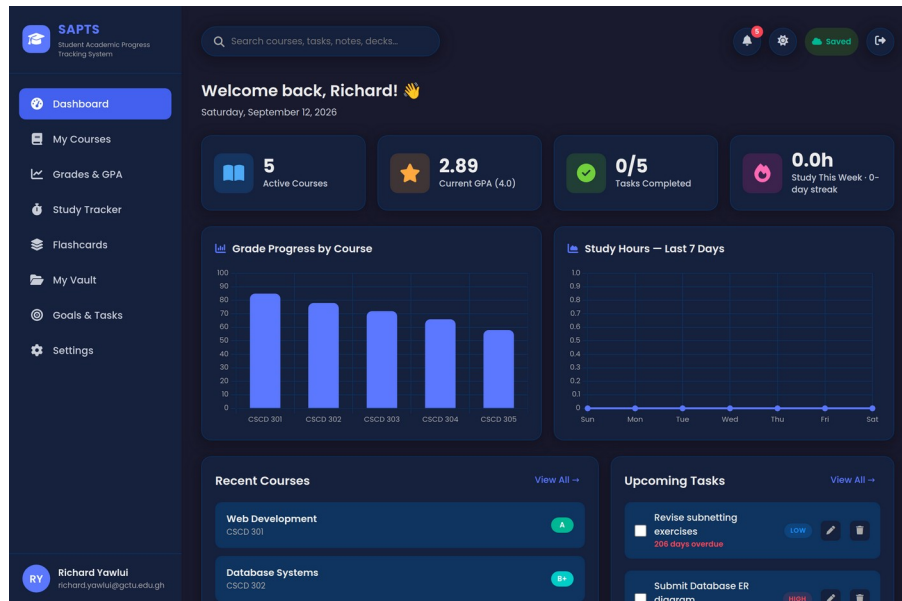


Figure 4.6: The dashboard in the dark theme

4.3 Conclusion

This chapter has reported the construction of the system, the testing it was subjected to and the measurements taken from it. Some 6,800 lines across fifteen files implement the design of Chapter Three, with the division between client and server governed by the rule that the client may calculate what it displays but is never trusted about what it stores. A scripted suite of twenty-eight cases, spanning authentication, authorisation, validation, calculation, persistence, layout and injection, passes in full; it earned that standing by first finding three real defects, each of which is recorded above with its cause and its remedy. Every non-functional requirement was measured and met, with the dashboard reaching an interactive state in 500 milliseconds against a three-second budget. The one qualification is that usability was established by a developer walkthrough rather than by a study with independent participants, and that shortfall is carried forward honestly into the discussion that follows.

CHAPTER FIVE

CONCLUSION AND RECOMMENDATIONS

5.1 Discussion

This chapter sets the completed system against the problem it was built to address, against the solutions that already existed, and against the objectives declared at the outset. It also records what proved difficult, what was learned, and what a subsequent worker should do next.

5.1.1 Comparison with Existing Systems

Chapter Two examined five categories of existing solution and found that each satisfied some of the criteria drawn from the problem statement and none satisfied all four. Table 5.1 places the delivered system alongside them against those same criteria.

Table 5.1: The delivered system against the solutions reviewed in Chapter Two

System	Integration	Effort to outcome	Academic specificity	Cost and access
MyStudyLife	Partial — scheduling only	No	Yes	Account and network required
Todoist and general planners	Partial — tasks only	No	No	Free tier; network required
Grade calculators	No — stateless	No	Yes	Free; nothing retained
Institutional LMS and SIS	Partial — marks only	No	Yes	Credentials and network required
Learning analytics dashboards	Yes	Partial	Yes	Institutional deployment only
This system	Yes — all functions	Yes	Yes	Free; runs offline on one machine

The distinction worth drawing is not that the system does more than every alternative. A learning analytics platform of the kind Arnold and Pistilli (2012) describe performs prediction this system does not attempt, and an institutional student information system holds the authoritative record, which this system explicitly does not. The distinction is that the combination on offer here existed nowhere: a student can

see their standing, the effort behind it and the work still outstanding in one place, at no cost, on a machine their department controls, without a network connection and without their academic record being held by a third party.

The comparison also exposes a genuine limitation. Because marks are entered by hand, the system holds what the student believes their standing to be rather than what the registry has ratified. It is an instrument for self-management, not a system of record, and it should not be presented as one.

5.1.2 Challenges Faced

Four difficulties were substantial enough to change the shape of the work.

- Moving from browser storage to a database. The system was first built to hold everything in the browser. Introducing accounts made a server unavoidable, and the awkwardness was that the interface read its data synchronously while a server responds asynchronously. Rewriting every module to await its data would have been extensive and risky. The resolution was to copy the signed-in student's records into browser storage once at sign-in and write every change back, which left the storage layer synchronous and the modules untouched.
- Two representations of the same rule. The grading scale exists twice, once in JavaScript for display and once in SQL for storage. The first attempt let the client send the letter grade it had computed, which introduced the defect described in section 4.1.3 and, worse, trusted the browser about a stored value. Having the server derive the letter independently cost a few lines and removed both problems.
- Responsive layout at phone width. The interface appeared to work on a narrow screen while in fact scrolling sideways, because a flex item will not shrink below its own content by default. The failure was invisible in casual use and only emerged when the document width was measured against the viewport programmatically. It is a reminder that a layout should be verified by measurement, not by looking at it.
- Deciding what the administrator may see. An administrator must be able to enrol a student and reset a forgotten password, both of which are powers over an account. It would have been easy to let the same console display academic records. It deliberately does not: it shows account status and a count of courses,

never a mark. The power to reset a password is unavoidable and is stated as a residual risk in section 3.3.4 rather than hidden.

5.1.3 Lessons Learned

The most useful lesson concerned the value of a single storage layer. Routing every read and write through one place looked like unnecessary ceremony while the data lived in the browser; it was what made the later move to a database a contained change rather than a rewrite. A boundary drawn before it is needed is cheap, and drawing it afterwards is not.

The second lesson concerned testing. Each of the three defects recorded in section 4.1.3 was invisible to inspection and obvious to a test: the data loss appeared only on a second round-trip, the overflow only when the width was measured, and the empty dashboard only on a genuinely fresh installation. Writing the suite as a script rather than as a checklist meant it could be re-run after every change, and its value came less from the passes than from what it caught.

The third concerned security. Decisions such as hashing passwords, regenerating the session identifier and binding every query to the session cost little when taken during design. Each would have been expensive to retrofit, and two of them, the timing of the failed sign-in and the refusal to accept an identifier from the request, are invisible in the interface and would never have been added later in response to a complaint.

The fourth concerned honesty about evidence. It is tempting, when a report calls for usability testing, to present a number. What was actually performed was a developer walkthrough, and it is reported as such in section 4.2.2. A claim that cannot be defended under questioning is worth less than an acknowledged gap.

5.1.4 Contributions to the Field

The project makes three modest contributions. The first is practical: a working system that GCTU students can use, which a department can install on a single machine without cost, licensing or an internet connection.

The second is a demonstration that the principles of self-regulated learning described by Zimmerman (2002) can be mapped onto software features concretely rather than rhetorically. Each phase of the cycle corresponds to a named module, and

Chapter Two traces the mapping explicitly, so the design can be argued from theory rather than from preference.

The third is a reference implementation. A complete multi-user system, with authentication, role-based authorisation, a normalised relational store and a documented security posture, delivered on the free XAMPP stack and accompanied by a reproducible test suite, is a useful template for subsequent projects in the Faculty working under the same constraints.

5.1.5 Limitations and Future Work

Four limitations are acknowledged. Marks are entered by hand, so the system reflects what a student records rather than what the institution has ratified. Usability was established by a developer walkthrough rather than by a study with independent participants. The system is served over plain HTTP on the local host, which is sound only because the client and server are the same machine. And an administrator can reset any student's password, so administrator access must be controlled as carefully as the records it protects.

Five lines of further work follow from these, in the order a subsequent worker should take them:

7. Conduct a usability study with a sample of students using the System Usability Scale, and revise the interface against what it finds. This is the outstanding evaluation and should come first.
8. Serve the system over HTTPS with a certificate, and harden the configuration, before it is placed on any network beyond the machine it runs on.
9. Import marks automatically from the institutional learning management system, which would remove manual entry and make the held record authoritative.
10. Add a lecturer role able to see aggregate, anonymised performance for a course, without access to any individual student's records.
11. Extend the analysis towards early warning, identifying courses at risk from the relationship between study effort and marks, in the manner of Arnold and Pistilli (2012).

5.2 Conclusion

This project set out to build a system that would let a university student see their academic standing, the effort behind it and the work still outstanding in one place, and to do so on infrastructure a department could own outright. Chapter One established that students lack such a system and that the fragmentation of their records has measurable consequences. Chapter Two grounded the response in self-regulated learning theory and showed that no existing tool combined the four properties required. Chapter Three specified thirteen functional and eight non-functional requirements and a three-tier design to satisfy them. Chapter Four reported the construction and the evidence that it works.

Each of the six objectives stated in section 1.3 has been met. The tracking needs were analysed and expressed as requirements. An intuitive, responsive interface was designed and verified to behave correctly from phone width to desktop. The core modules were developed and each was tested. A relational data layer with account-based access control was implemented, and the separation between students was demonstrated rather than asserted. An enrolment mechanism was provided and the system deployed on Apache and MySQL under XAMPP. And the system was tested for correctness, security and performance, with twenty-eight cases passing and every measurable requirement met.

Two things are worth stating without embellishment. The system is an instrument for self-management and not a system of record, because the marks within it are entered by the student. And its usability rests on a developer walkthrough rather than on independent participants, which is the first piece of work a successor should complete. Within those bounds, the system does what it was built to do, on a machine the department controls, at no cost, and without any student's academic record leaving the institution.

REFERENCES

- Arnold, KE and Pistilli, MD (2012) 'Course Signals at Purdue: using learning analytics to increase student success', Proceedings of the 2nd International Conference on Learning Analytics and Knowledge, 267-270
- Bharamagoudar, SR, Geeta, RB and Totad, SG (2013) 'Web based student information management system', International Journal of Advanced Research in Computer and Communication Engineering, 2:6, 2342-2348
- Black, P and Wiliam, D (1998) 'Assessment and classroom learning', Assessment in Education: Principles, Policy and Practice, 5:1, 7-74
- Brooke, J (1996) 'SUS: a quick and dirty usability scale', in Jordan, PW et al (eds) Usability Evaluation in Industry, Taylor and Francis, London, 189-194
- Cirillo, F (2018) The Pomodoro Technique: The Life-Changing Time Management System, Virgin Books, London
- Dunlosky, J, Rawson, KA, Marsh, EJ, Nathan, MJ and Willingham, DT (2013) 'Improving students' learning with effective learning techniques', Psychological Science in the Public Interest, 14:1, 4-58
- Ellis, RK (2009) Field Guide to Learning Management Systems, American Society for Training and Development, Alexandria
- Few, S (2012) Show Me the Numbers: Designing Tables and Graphs to Enlighten (2nd edn), Analytics Press, Burlingame
- Hattie, J and Timperley, H (2007) 'The power of feedback', Review of Educational Research, 77:1, 81-112
- Kruchten, P (1995) 'Architectural blueprints: the 4+1 view model of software architecture', IEEE Software, 12:6, 42-50
- Mozilla Developer Network (2024) IndexedDB API [online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API [accessed 12 August 2026]
- Mozilla Developer Network (2024) Web Storage API [online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API [accessed 12 August 2026]
- Nielsen, J (1994) Usability Engineering, Morgan Kaufmann, San Francisco
- Pintrich, PR (2004) 'A conceptual framework for assessing motivation and self-regulated learning in college students', Educational Psychology Review, 16:4, 385-407
- Pressman, RS and Maxim, BR (2020) Software Engineering: A Practitioner's Approach (9th edn), McGraw-Hill, New York
- Roediger, HL and Karpicke, JD (2006) 'Test-enhanced learning: taking memory tests improves long-term retention', Psychological Science, 17:3, 249-255
- Royce, WW (1970) 'Managing the development of large software systems', Proceedings of IEEE WESCON, 1-9

- Sarker, F, Davis, H and Tiropanis, T (2010) 'A review of higher education student data use for learning and teaching', Proceedings of the International Conference on Computer Supported Education, 1-8
- Shneiderman, B and Plaisant, C (2010) Designing the User Interface: Strategies for Effective Human-Computer Interaction (5th edn), Addison-Wesley, Boston
- Siemens, G and Long, P (2011) 'Penetrating the fog: analytics in learning and education', EDUCAUSE Review, 46:5, 30-40
- Sommerville, I (2016) Software Engineering (10th edn), Pearson Education, Harlow
- The Apache Friends (2024) XAMPP Apache + MariaDB + PHP + Perl [online]. Available: <https://www.apachefriends.org> [accessed 12 August 2026]
- Zimmerman, BJ (2002) 'Becoming a self-regulated learner: an overview', Theory Into Practice, 41:2, 64-70
- Zimmerman, BJ and Schunk, DH (2011) Handbook of Self-Regulation of Learning and Performance, Routledge, New York